# Contents



# UX for Role-Based Platforms — A Working Checklist

*Last updated: 2026-08-12*

**What this is for:** a reusable review checklist for any multi-role product, plus the reasoning behind it. Run Part I against a screen before shipping it. Part II is a worked case study (Project Mentor: track vs team surfaces) showing the checklist producing an actual design decision. Part III distils the principles worth carrying to a new system.

**Companion notes:** eng-labs-platform — the system's architecture. multi-tenancy — isolation patterns and codebase lessons.

---

# Part I — The 13 checks

The spine of this list is mine; what follows each item is how to actually test it and where eng-labs passes or fails.

### 1 · Roles — who is this user, and what does this role actually need?

Write the role's **job**, not its permissions. "An instructor can edit tracks" is a permission; "an instructor needs to know which teams are behind" is a job. Screens designed from permissions become feature dumps.

> **eng-labs has three overlapping role systems** — platform (`ADMIN`/`INSTRUCTOR`/`LEARNER`/`COORDINATOR`), per-track (`PROJECT_INCHARGE`/`PROJECT_MENTOR`/`LEARNER`), and module access (`PM_ADMIN`/`PM_MEMBER`). A professor can be incharge on one track and mentor on another, so **role is per-context, never per-user.** Any UI that reads "the user's role" globally is already wrong.

### 2 · Reach — can they get to what they want in a few clicks?

Count the clicks on the **most frequent** path, not the average one. Three clicks to a rare admin setting is fine; three clicks to the thing done fifty times a day is a design failure.

Test: name the top 3 tasks per role, then count. If any is >2 clicks from landing, it needs a shortcut or a different landing screen.

### 3 · Find — is the stuff used most front-and-centre, not buried?

Frequency should decide position. A tab that exists because the data model has a table is the classic anti-pattern.

eng-labs example of getting this right: **Notifications was removed from track level and team level and kept only at root.** Same data, one home, no nesting.

### 4 · Land — does each role open onto the screen that matters to them?

The landing screen is the strongest signal you send about what the product is for. Every role gets its own.

Learner lands on recent projects + activity. Instructor lands on a dashboard, with **My Teams** and **Review Queue** one click away. Nobody lands on a generic "welcome."

### 5 · Show-only-allowed — hide or disable what they can't do

Never render a button whose only behaviour is to say "denied." Prefer *absent* over *disabled* for things the role will never gain; use *disabled + reason* when the block is temporary ("submissions open on 12 Sep").

**Critical caveat — hiding is UX, never security.** A prior eng-labs review found the frontend hid buttons while the APIs stayed open. Hidden UI plus an open endpoint is a vulnerability with a polite face. The server enforces; the UI only informs.

### 6 · Feedback — does every action say loading / done / failed?

Every mutation needs three states, and the success message should name what happened ("Published" after pressing Publish), not "Success."

Test: click every button with the network throttled. Anything that looks inert for >400ms without a spinner will be clicked twice by a real user.

### 7 · Helpful errors — say what went wrong and what to do next

Two parts, both required: the cause, and the next action. "Permission denied" fails; "Only the track incharge can publish — ask [name] to publish this track" passes.

eng-labs has good bones here: `errorHandler(ERROR_CODES.ACCESS_DENIED, "...")` centralises the shape so messages can be improved in one place. The gap is that many messages are still generic.

### 8 · Empty states — guide the next step, don't look broken

An empty screen is a teaching opportunity and the most-neglected screen in every product. Three ingredients: what this screen is for, why it's empty, and the one action that fills it.

"No teams yet" is a bug report. "No teams yet — teams appear here once students form groups, or create them yourself →" is onboarding.

**9 · Fast frequent flows — quick, not merely reachable**

Reachable (#2) and fast are different. A mentor reviewing 10 submissions shouldn't re-navigate between each one. This is where **sequential** flows need their own surface.

> This is exactly what the Review Queue exists for — see Part II.

**10 · Scale — can they find one item among thousands?**

Every list must answer: what happens at 10, 1,000, and 100,000 rows? Search, filter, sort, paginate — and server-side, not client-side, past a few hundred.

> A track can hold thousands of students. Any student list without server-side search is a screen that works in the demo and dies at the customer.

**11 · Their words — labels match the user's vocabulary, everywhere**

The user's noun, not the schema's. And **the same noun for the same thing on every screen.**

> The strongest eng-labs example: `project_tracks` is one table shown under two names. Learners and mentors see **"Project"**; the incharge sees **"Track"**. Chosen per-track from `myEnrollment.role`, because the same professor is incharge on one and mentor on another.
>
> Crucially this is **UI-only** — no table, column, model, route or store key was renamed. Relabelling the wire and the schema to match the UI is how you get a two-year migration for a vocabulary fix.

**12 · Speed — heavy screens load fast, or feel broken**

Perceived speed is the product. Skeletons over spinners; paginate; and fix N+1 queries, which are the usual culprit behind "the dashboard is slow."

> eng-labs precedent: an analytics overview was rewritten from N+1 queries to batch fetching + in-memory joins. The faster version was also the shorter one.

**13 · Handoffs — when work passes between roles, everyone sees status**

The most-missed check, and the one users complain about most. For every handoff, ask: does the sender know it arrived? Does the receiver know it's waiting? Can both see where it is?

> The learner→mentor submission handoff is the core loop of Project Mentor. It needs: learner sees "submitted, awaiting review", mentor sees it in the Review Queue, both see the state change when feedback lands.

**Two checks I'd add to your list**

**14 · Undo over confirm.** A confirmation dialog trains people to click through. An undo affordance actually protects them. Reserve modals for the genuinely irreversible.

**15 · Deep-linkable state.** If a user can see it, they should be able to link to it. This sounds like polish and isn't — it's what makes notifications, review queues, and "can you look at this?" messages work at all. See Part II §3.

---

## Part II — Case study: Project Mentor's track vs team surfaces

The question: *an instructor is incharge of a track and also mentors 10 of its teams. Where does mentoring work live?*

### 1 · The data model already answered it

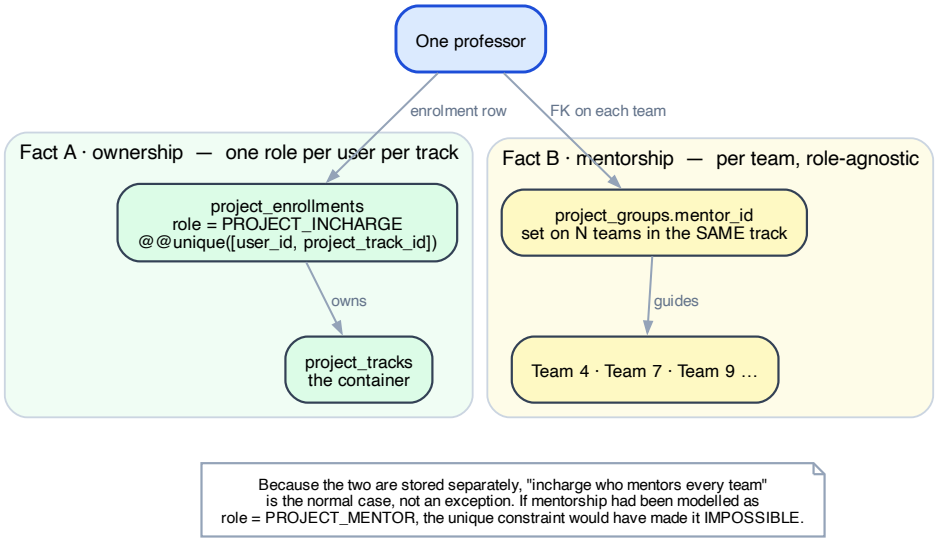


Figure 1: Ownership and mentorship are orthogonal facts

| Fact | Stored as | Cardinality |
|---|---|---|
| "I own this track" | `project_enrollments.role = PROJECT_INCHARGE` | **one role per user per track** (`@@unique([user_id, project_track_id])`) |
| "I mentor this team" | `project_groups.mentor_id` | per team, independent |

Because these are separate, "incharge who mentors every team" is the **normal** case, not an exception. The codebase already knows this — `apps/api/src/routes/projects/admin-dashboard.ts:316`:

```
// "Mentor" = whoever is the team's mentor_id (role-agnostic), so an incharge
// acting as mentor is counted.
```

**The near-miss worth remembering:** had mentorship been modelled as `role = PROJECT_MENTOR` in enrolments, the unique constraint would have made the overlap **inexpressible**. Storing a relationship on the entity it belongs to (`mentor_id` on the team) rather than as a role on the container is what kept it possible.

> **Transferable rule:** before splitting a UI by role, check whether the roles are mutually exclusive *in the data*. If they overlap, a role-shaped UI will break at exactly the overlap.

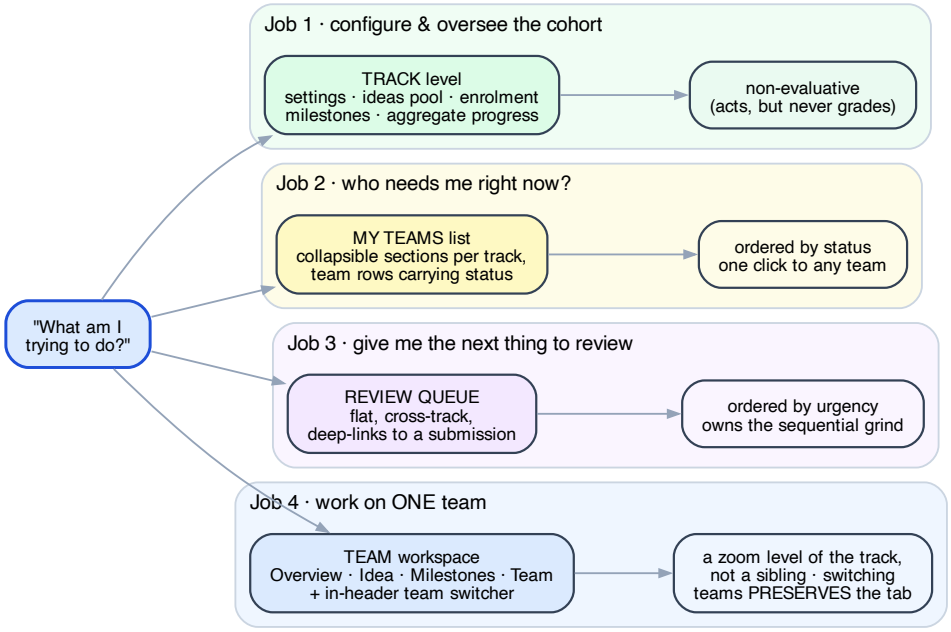


Figure 2: Surfaces mapped to jobs, not roles

## 2 · Organise by job, not by role

| Surface | Job | Ordered by |
|---|---|---|
| **Track level** | configure and oversee the cohort | config + aggregate + exceptions |
| **My Teams list** | "who needs me right now?" | status, grouped by track |
| **Review Queue** | "give me the next thing to review" | urgency, across all tracks |
| **Team workspace** | work on one team | the team's own timeline |

These stay four different jobs even when one person does all four. Splitting by role instead would collapse the moment someone holds two — which is the common case.

**Track level is *non-evaluative*, not read-only.** The incharge genuinely acts there: publish, configure milestones, assign mentors, handle stranded students. What it must not host is per-submission grading — not for permission reasons, but because a grading UI doesn't survive being rendered 20 times, and two places to write feedback means two places to look for it.

> **The one hard rule:** evaluation happens at team scope only, for everyone, including the incharge who mentors every team.

## 3 · Group, don't nest — and route, don't filter

The proposed My Teams flow was: track card → click → filter teams → team. Two refinements:

**Group instead of nesting.** One list with collapsible track sections beats a card you must click through:

```
DBMS Capstone 2025-26            10 teams · 3 awaiting review
    Team 4 · Airline DBMS           2 submissions pending
    Team 7 · Library System         up to date
    Team 9 · Hotel Booking          milestone overdue
AI Systems 2025-26                4 teams · 1 awaiting review
```

One click to any team instead of two. Auto-expand when there's a single track; collapse when there are several. Degrades well at both 10 teams in one track and 30 across four.

**The row must carry status, not just a name.** A row reading only "Team 7" doesn't answer the question the mentor arrived with. Without status, this screen is a directory and mentors will live in the Review Queue instead — making it decorative.

**Route, don't filter.** The selected team must be in the URL, not component state:

`/projects/[trackId]/teams/[teamId]/milestones`

Filter-as-state silently breaks deep-linking from the Review Queue and from notifications, bookmarking, the back button, and "send me the link." *Present* it as a switcher; *implement* it as a route. Those aren't in conflict — and this is check #15.

## 4 · The in-team switcher earns its place on exactly one behaviour

Keep a team switcher in the team workspace header — but only because of this:

> **Switching team preserves the current tab.** `/projects/t1/teams/4/milestones` → switch → `/projects/t1/teams/9/milestones`

The list always drops you at a team's default tab. A mentor checking milestones across teams wants to hold the *task* constant and change the *team*. Without tab persistence the switcher is a slower back button and should be cut.

Rules: scope it to teams **I** mentor in this track (all-teams is the incharge's view); carry the same status glyphs as the list; hide it when there's one team.

**And an honest scoping call:** the switcher is for *monitoring and lateral browsing*, not for grinding through reviews — the Review Queue owns that, because it's ordered by what needs action rather than by team. So build the switcher simple and don't invest further until mentors actually use it. If scope must be cut, the order is Review Queue → My Teams list → switcher.

## 5 · Two doors to the same place need different subtitles

An incharge who mentors sees the same track in both My Tracks and My Teams. That's honest — two roles — but reads as a duplicate unless labelled:

```
My Tracks →  DBMS Capstone 2025-26   Track · 20 teams · you are incharge
My Teams  →  DBMS Capstone 2025-26   10 teams you mentor · 3 awaiting review
```

Same destination once you drill into a team, which is fine — by then the ambiguity is gone.

### 6 · Tab parity is a decision, not a detail

If team tabs are meant to be identical for mentor and learner, the mentor's grading controls must live **inside the Milestones tab**, rendered when `mentor_id === me` — not as a mentor-only fifth tab. Decide this explicitly: "extra affordances on a shared tab" and "an extra tab" are similar to build and very different to undo.

---

## Part III — The transferable principles

1. **Organise surfaces by job, not by role.** Roles overlap; jobs don't. Role-shaped navigation fails precisely at the users who hold two roles — usually your most important users.
2. **Check role exclusivity in the data before designing around it.** If two roles can coexist on one row, the UI must assume they will.
3. **Store a relationship on the entity it belongs to,** not as a role on the container. `mentor_id` on the team, not `PROJECT_MENTOR` on the enrolment.
4. **Non-evaluative read-only.** Separate *what a surface is for* from *what a user is permitted to do*. Conflating them produces either a crippled overview or a grading panel in the wrong place.
5. **One place to perform an action.** Two places to write feedback = two places to look for it.
6. **Group, don't nest.** A screen whose only content is one card is a wasted click.
7. **Every list row answers the question the user arrived with.** Names are not status.
8. **If a user can see it, they can link to it.** State that isn't in the URL can't be linked, notified, or bookmarked.
9. **Zoom level, not sibling.** Breadcrumb `Track › Team 7`, so drilling in never feels like leaving.
10. **Relabel the UI, never the schema.** Vocabulary fixes should cost a string map, not a migration.
11. **Hiding is UX; the server enforces.** Hidden UI over an open endpoint is a vulnerability that looks tidy.
12. **Sequential work needs its own surface.** Browsing and grinding are different jobs; a queue is not a list.